

Assignment 1

Due Wednesday, February 12

The assignment contains three parts. Part I (the common part for both 4MA433 and 4MA923) consists of a written part and an implementations part. Part II (only for 4MA433) is the theoretical part. It contains more theoretical problems sometimes depending on knowledge of abstract algebra. Part III (only for 4MA923) is the computational part. It contains more advanced programming problems.

- It is fine to work together as long as you write and understand your own solutions. (No copying!)
- To each assignment each student should submit three files on Moodle:
 1. Part I (written part) as a pdf-file. You can use L^AT_EX for this part, but it is also fine to submit scanned handwritten solutions. You are responsible for these being readable.
 2. Part I (implementation part) as a Python file (.py) or a Jupyter file(.ipynb)
 3. Students in 4MA433 submit Part II as a pdf-file. Students in 4MA923 submit Part III as a Python- or Jupyter-file.
- On the seminars the problems of the Part I is presented and discussed. Part II and Part III are discussed on the oral exam in the end of the course.
- Part I is graded according to a linear scale. To pass (grade E) 50% is needed, for grade C, 70% is needed and for grade A, 90%. Part II and Part III is graded together with the oral exam.

Part I (common part)

Implementation part

1. Let us implement the Affine Cipher. Write two functions

```
affineEncrypt(P,a,b,m)
affineDecrypt(C,a,b,m)
```

where P is the encoded plaintext, C is the ciphertext (encoded), a , b and m are the numbers used in the cryptosystem with the encryption function $e(P) \equiv aP + b \pmod{m}$. When construction the decryption function you should call the function `multInverse` from Assignment 0.

- (a) Choose a string that contains at most 16 characters. It can for example be part of your name. Encode it to a number by using the `textToInt`-function. This will be your plaintext P .
- (b) Let $m = 340282366920938463463374607431768211507$ (this is a prime). Choose two numbers a and b in $(\mathbb{Z}/m\mathbb{Z})^*$. Encrypt your message P by using `affineEncrypt` and save the ciphertext you got. Try now to decrypt the ciphertext by using `affineDecrypt` and decode it using `intToText`. Did you recover your message?
- (c) What happens if you start with a message containing 20 characters?

- (d) I have used the same value of m as above, but I have used $a = 21712651827515182$ and $b = 274812741241924192499$. Decrypt (and decode) the following ciphertext:

306223679614151152019567562383624433868

Remember to not believe everything you read!

2. Alice and Bob are going to agree on a key for a symmetric cryptosystem. They will use the Diffie-Hellman key exchange protocol. Show that they will arrive to the same key given the parameters below.

- (a) Let $p = 2712691$, $g = 2$, $a = 2553$ and $b = 26511$.
 (b) Let $p = 340282366920938463463374607431768211507$,
 $g = 121770841829326452690259895862715513623$, $a = 21712651827515182$
 and $b = 274812741241924192499$.

3. We will implement the Elgamal public key cryptosystem.

- (a) Write two functions `elgamalEncrypt(p,g,A,m)` and `elgamalDecrypt(p,g,a,C)`, where p is a prime, g is the generator of a subgroup in $\mathbb{F}_p^*$, A is the public key, a is the corresponding private key, m is the message and $C = [c1, c2]$ is the ciphertext produced by the encryption function. Use the functions `fast2Power` and `multInverse` from Assignment 0. To generate the ephemeral key you can use `randint(x,y)` from the random module. It will give you a number n s.t. $x \leq n \leq y$.
 (b) Let $p = 2189248127867$ and $g = 1267362$. Let the private key be $a = 28127481748$. Compute the public key. Use it to encrypt a message of five characters length by using `elgamalEncrypt`. Use the `textToInt` function.
 (c) Decrypt the ciphertext produced in (b) and use the `intToText`-function. Did you recover the message?
 (d) I have used the Elgamal cryptosystem with $p = 286197821500073589067075140527$, $g = 195051999861158821970552089016$ and your public key (corresponding to your private key $a = 7712654356741$) to encrypt a message. The ciphertext is

$C = [223905580785475751236633059935, 197494619985708086124722797272]$

Find the plaintext!

4. We will now implement Elgamal's digital signature algorithm.

- (a) Write a function `generateElgamalSig(p,g)` where p is a so called safe prime of the form $p = 2q + 1$, where q is a prime and g has period q . The output will be a private signing key a , and a public verification key A .
 (b) Write the function `signElgamal(a,p,g,D)` that signs the document D and the function `verifyElgamal(A,p,g,D,S)` that verifies that S is the signature of D .
 (c) Use Elgamal's signature algorithms with a 256-bit prime
 $p = 10368946271683894726408735961479164250456225873359395592239263831895651207951$
 9

and

$g = 38800622773637695321106177828004110206779343423539510992140900880451899659111$

Sign a document D (of your choice) such that $1 < D < p$ and verify that the signature is correct.

5. Solve the following DLP by using Shank's Babystep-Giantstep algorithm, see the `babyGiant` function in the end of this document.

(a) $2^x \equiv 2861 \pmod{3079}$

(b) $g^x \equiv h \pmod{p}$, where $p = 270826551115777$, $g = 126736225125251$ and $h = 159906865261704$.

Written part

6. Solve this problem without any computational aid. Let $p = 23$.

(a) Show that 10 is a primitive root modulo 23.

(b) Find $\log_{10}(14)$.

7. Let $p = 2189248127867$. Find the order of $a = 1267362$ modulo p . (You might use the function `factorint` from the `sympy` module and your `fast2Power`-function.) Note also that the order must be a divisor of $p - 1$.

8. It is especially hard to solve the equation $g^x \equiv h \pmod{p}$ if the order of g is a (large) prime. The best primes are on the form $p = 2q + 1$ where also q is a prime. Such primes are called *Safe primes* and the corresponding q is called a *Sophie Germain prime*.

(a) Show that if $p = 2q + 1$ (where q is a prime) then $p \equiv 5 \pmod{6}$ or $p = 7$. Hint: Consider three cases $q = 3k$, $q = 3k + 1$ and $q = 3k + 2$

(b) Let $g \in \mathbb{F}_p^*$. Which are possible orders of g ? How many elements in $\mathbb{F}_p^*$ have order q ? Hint: Recall that $\text{ord}(g^k) = \frac{p-1}{\gcd(k, p-1)}$.

(c) Let $p = 3023$ (a Safe prime). One can prove that 5 is a primitive root modulo p . Find $g \in \mathbb{F}_{3023}^*$ such that the order of g is 1511.

9. Solve the DLP

$$2^x \equiv 2861 \pmod{3079}$$

by using Pohlig-Hellman algorithm.

Part II (Theoretical part)

Only for students in 4MA433.

1. Let p be a prime and let g be a primitive root modulo p . Let $a \in \mathbb{F}_p^*$.

(a) Let m be the smallest positive number such that $a^m \equiv 1 \pmod{p}$. Then m is called the order of a modulo p . Show that $m \mid (p - 1)$.

- (b) Let $k \in \{0, 1, \dots, p-2\}$. Prove that the order of g^k must divide $\frac{p-1}{\gcd(k, p-1)}$. *Hint:* Let $d = \gcd(k, p-1)$ and compute $(g^k)^{\frac{p-1}{d}}$.
- (c) Prove that $\frac{p-1}{d}$ divides the order of g^k . *Hint:* Compute $(g^k)^{\text{ord}(g^k)}$, where $\text{ord}()$ denotes the order.
- (d) Conclude that $\text{ord}(g^k) = \frac{p-1}{\gcd(k, p-1)}$.
2. Let $a, b \in \mathbb{Z}$. Let $m \in \mathbb{Z}^+$. Show that $ax \equiv b \pmod{m}$ has a solution if and only if $\gcd(a, m) \mid b$. If the equation has solutions show that it has exactly $\gcd(a, m)$ solutions modulo m . *Hint:* Let $d = \gcd(a, m)$. If $d \mid b$, use the extended Euclidean algorithm to find one solution x_0 . Show that all the solutions modulo m are

$$x_0, x_0 + 1 \cdot \frac{m}{d}, x_0 + 2 \cdot \frac{m}{d}, \dots, x_0 + (d-1) \frac{m}{d}.$$

3. Samantha is using the Elgamal digital signature algorithm. In this problem you will investigate what might happen if she use the same random number k to sign two different documents D and D' .
- (a) How can Eve recognize that Samantha have done this mistake?
- (b) Let (S_1, S_2) be the signature of D and let (S'_1, S'_2) be the signature of D' . Explain how Eve can recover Samantha's secret signing key a . *Hint:* Study the difference $S_2 - S'_2$ and use problem 2 to find k^{-1} . Then use the definition of S_2 (and Problem 5 again) to find a .
4. Let p be a prime and let $p-1 = m_1 \cdot m_2 \cdots m_k$, where all the m_i are pairwise relatively prime. When using Pohlig-Hellman to solve $g^x \equiv h \pmod{p}$ we computed local solutions to $g_i^{x_i} \equiv h_i \pmod{m_i}$, where

$$g_i = g^{\frac{p-1}{m_i}} \text{ and } h_i = h^{\frac{p-1}{m_i}},$$

for all $1 \leq i \leq k$. We use the Chinese Remainder Theorem to find an x satisfying $x \equiv x_i \pmod{m_i}$. Show that this x solves the original problem $g^x \equiv h \pmod{p}$. *Hint:* Since the greatest common divisor of $\frac{p-1}{m_i}$ is 1 we can by repeated use of the extended Euclidean algorithm find integers c_i such that

$$c_1 \frac{p-1}{m_1} + c_2 \frac{p-1}{m_2} + \dots + c_k \frac{p-1}{m_k} = 1$$

Show that $c_i \equiv \left(\frac{p-1}{m_i}\right)^{-1} \pmod{m_i}$.

Part III (Computational part)

Only for students in 4MA923.

1. Implement the Digital Signature Algorithm (DSA). (See HPS Section 4.3.)
- (a) Write a function `generateDSA(p,q,g)` where p and q are primes such that $p \equiv 1 \pmod{q}$ and $g \in \mathbb{F}_p^*$ of order q . It should give a secret signing key a and a public verification key A .
- (b) Write also the functions `signDSA(a,p,q,g,D)` (a is the signing key, D is the document) and `verifyDSA(A,p,q,g,S,D)`, where A is the verification key and S the signed document.

- (c) Let
 $p = 77435081224609491566246186553934541650038467378511547335936824568950781244967$,
 $q = 16496047420474056593$ and
 $g = 52438402441471243928180899968147631418009321449216573273888371406197177211989$
and run your `generateDSA` function.
- (d) Sign the following message
I <YOUR NAME> hereby certify that I have done this project myself!
(Replace <YOUR NAME> by your own name.) First use your `textToInt`-function then hash the message by the NSHA (Not Secure Hash Algorithm) $h(M) = M \bmod q$.
- (e) Verify your own signatures by using your `verifyDSA`-function.
2. Implement the Pohlig-Hellman algorithm by constructing a function `pohligHellman(g,h,p,N)`, where N is the order of g . You should use the function `factorint` from `sympy` and the functions `fast2Power`, `multInverse` from Assignment 0 and `babyGiant` from this assignment.
- (a) Let $p = 270826551115777$, $g = 126736225125251$ and $h = 159906865261704$. The order of g is $N = 90275517038592$. Find $\log_g(h)$. Compare the answer when you just used Babystep - Giantstep algorithm. Comments?
- (b) Let $p = 286197821500073589067075140527$, $g = 195051999861158821970552089016$ and $h = 232260532812121798828459018892$. Find $\log_g(h)$. Did it work?
3. Eve has understood that Bob is communicating with Alice by using the Elgamal cryptosystem with the public keys:

$$\begin{aligned}p &= 2023306774316798283884543108953, \\g &= 15933608878580910051027282678, \\A &= 507161866845660423833984216685.\end{aligned}$$

Eve manage to find the following ciphertext Bob sends to Alice

$$C = [62253865849879932462212496812, 624465978997833286127272847821].$$

Help Eve to find the text message Bob sent to Alice. What is Alice private key?

Some Algorithms

```
1 def babyGiant(g,h,p,N=-1):
2     if N==-1: #If we do not know the order we use N=p-1
3         N=p-1
4     n=int(sqrt(N))+1
5     gb=1
6     i=0
7     babyStepTable={gb:i}
8     while i<n-1:
9         gb=(gb*g)%p
10        i=i+1
11        babyStepTable[gb]=i #Add a new element to the table
12    G=fast2Power(multInverse(g,p),n,p)
13    y=h
14    if y in babyStepTable:
15        return babyStepTable[y]
16    for j in range(1,n):
17        y=(y*G)%p
18        if y in babyStepTable:
19            return j*n+babyStepTable[y]
20    return None #If no match is found
```